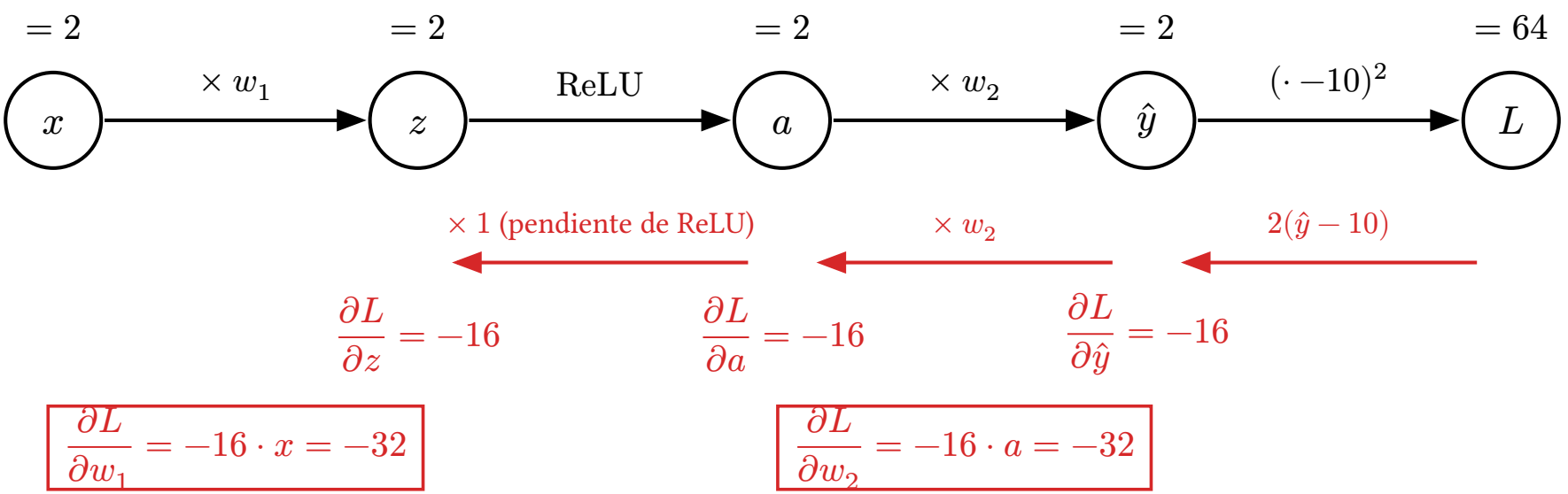
# Redes convolucionales

Tomas Santana

November, 2026

# Repaso: backpropagation



# Ejercicio 1

$$z = w_1 x$$
$$a = \mathrm{ReLU}(z)$$
$$\hat{y} = w_2 a$$
$$L = (\hat{y} - y)^2$$

Con $x = 1$, $y = 4$, $w_1 = 2$ y $w_2 = 3$:

1. Calcular el forward pass: $z$, $a$, $\hat{y}$ y $L$.
2. Calcular, de derecha a izquierda, las derivadas de $L$ respecto a $\hat{y}$, $w_2$, $a$, $z$ y $w_1$.
3. Dar un paso de descenso por gradiente con $\alpha = 0.01$. ¿Los pesos suben o bajan? ¿Por qué?

# backward() en PyTorch

```
x, y = 2.0, 10.0
w1 = torch.tensor(1.0, requires_grad=True)
w2 = torch.tensor(1.0, requires_grad=True)

z = w1 * x                    # forward pass
a = torch.relu(z)
y_hat = w2 * a
L = (y_hat - y) ** 2

L.backward()                  # backpropagation
print("L =", L.item())
print("dL/dw1 =", w1.grad.item())
print("dL/dw2 =", w2.grad.item())
```

**Salida**

```
L = 64.0
dL/dw1 = -32.0
dL/dw2 = -32.0
```

# Una red como clase: `nn.Module`

```
class MyMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(784, 128)
        self.out = nn.Linear(128, 10)

    def forward(self, x):
        x = x.flatten(start_dim=1)  # (N, 1, 28, 28) -> (N, 784)
        x = torch.relu(self.hidden(x))  # (N, 128)
        return self.out(x)  # (N, 10): logits, sin softmax
```

# Usar la red

```
if __name__ == "__main__":
    model = MyMLP()
    x = torch.rand(64, 1, 28, 28)  # un mini-batch de 64 imágenes
    print(model(x).shape)
    print(sum(p.numel() for p in model.parameters()))
```

Salida

```
torch.Size([64, 10])
101770
```

# Entrenar por mini-batches

```
def train(model, X_train, y_train, X_val, y_val, num_epochs=3, lr=0.1):
    datos = DataLoader(TensorDataset(X_train, y_train),
                       batch_size=64, shuffle=True)
    loss = nn.CrossEntropyLoss()
    optimizer = torch.optim.SGD(model.parameters(), lr=lr)
    for epoca in range(num_epochs):
        model.train()                # modo entrenamiento
        for X_batch, y_batch in datos:  # un paso por mini-batch
            optimizer.zero_grad()
            L = loss(model(X_batch), y_batch)
            L.backward()
            optimizer.step()
        acc = accuracy(model, X_val, y_val)
        print(f"época {epoca + 1}: validación {acc:.3f}")
```

# Ejercicio 2

50 000 ejemplos
mini-batches de 64
3 épocas

1. ¿Cuántos pasos da el entrenamiento?
2. ¿Y si los mini-batches son de 128?
3. ¿Cuántas veces ve la red cada ejemplo en los dos casos?

# Evaluar el modelo

```python
def accuracy(model, X, y):
    model.eval()                # modo evaluación
    with torch.no_grad():       # sin registrar operaciones
        predicted_classes = model(X).argmax(dim=1)
    return (predicted_classes == y).float().mean().item()
```

**MyMLP en MNIST**

```
época 1: validación 0.914
época 2: validación 0.937
época 3: validación 0.952
```

# El problema de las imágenes



un 4

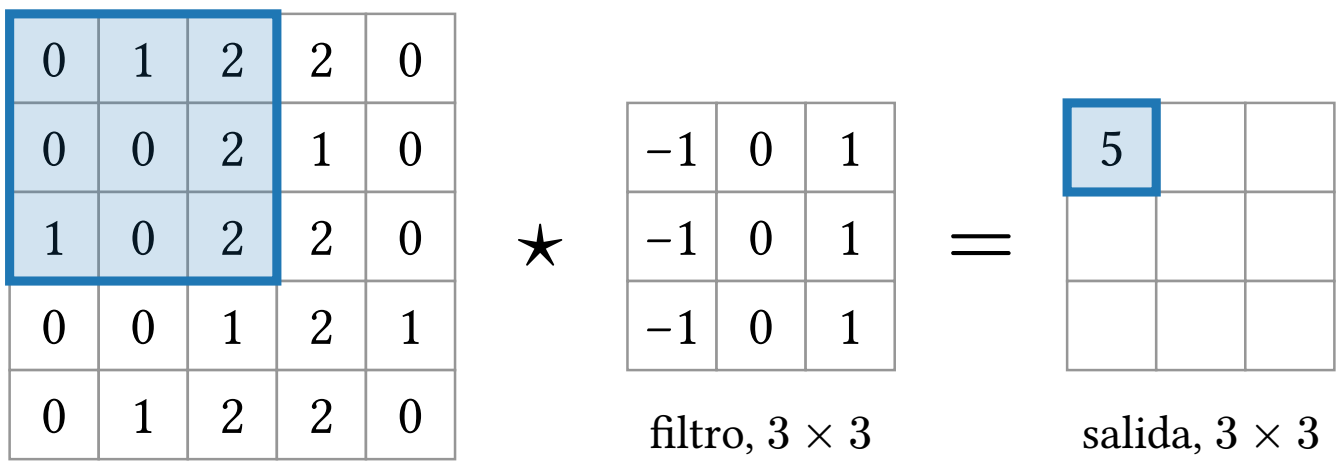


el mismo 4, desplazado
4 píxeles
distancia al primero:
**2 196**



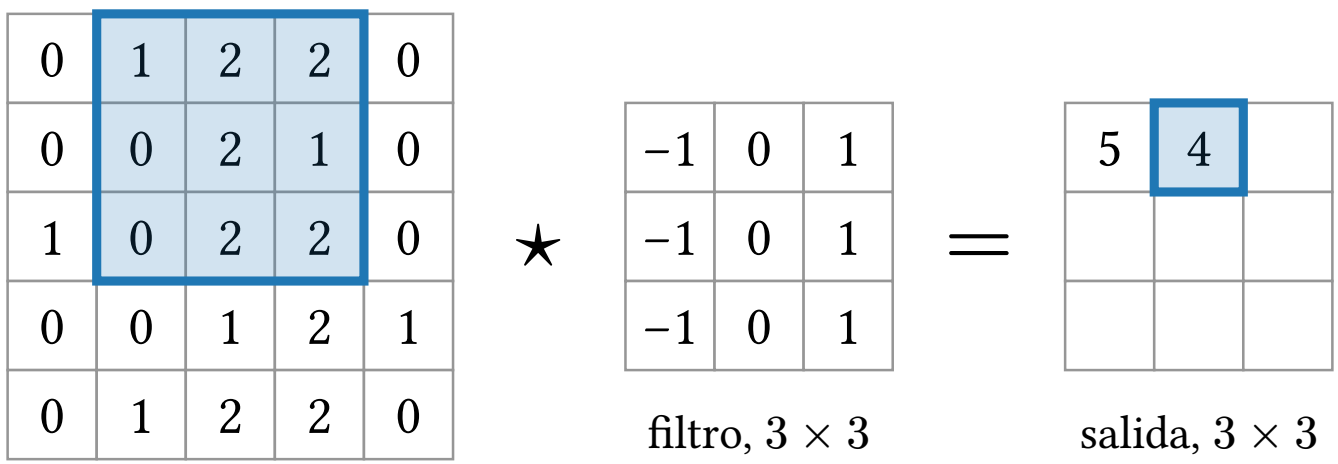
el 9 más parecido
al primer 4
distancia al primero:
**1 027**

# La convolución



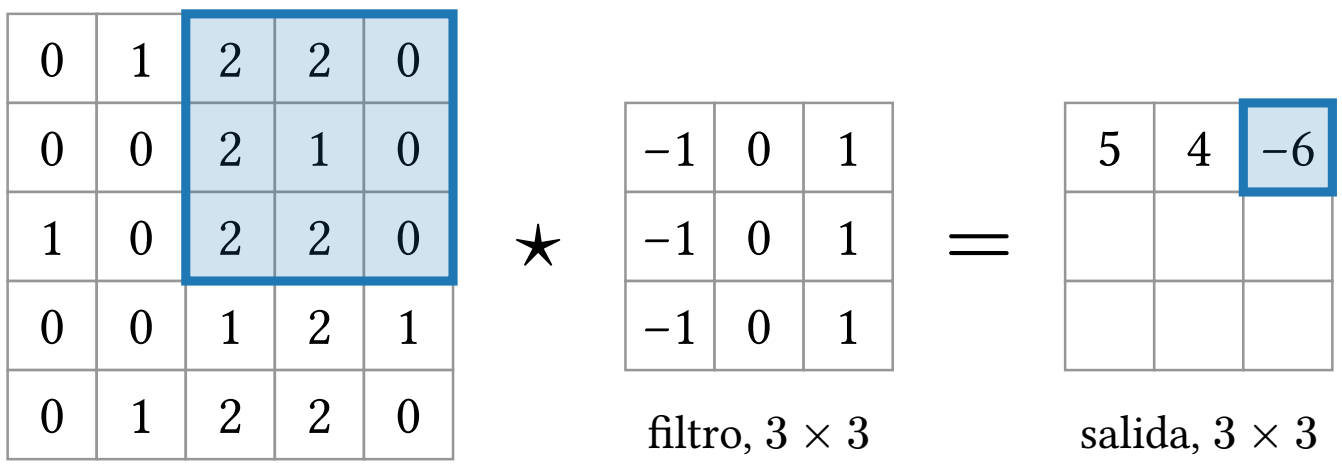

$$0 \cdot (-1) + 1 \cdot 0 + 2 \cdot 1 + 0 \cdot (-1) + 0 \cdot 0 + 2 \cdot 1 + 1 \cdot (-1) + 0 \cdot 0 + 2 \cdot 1 = 5$$

# La convolución



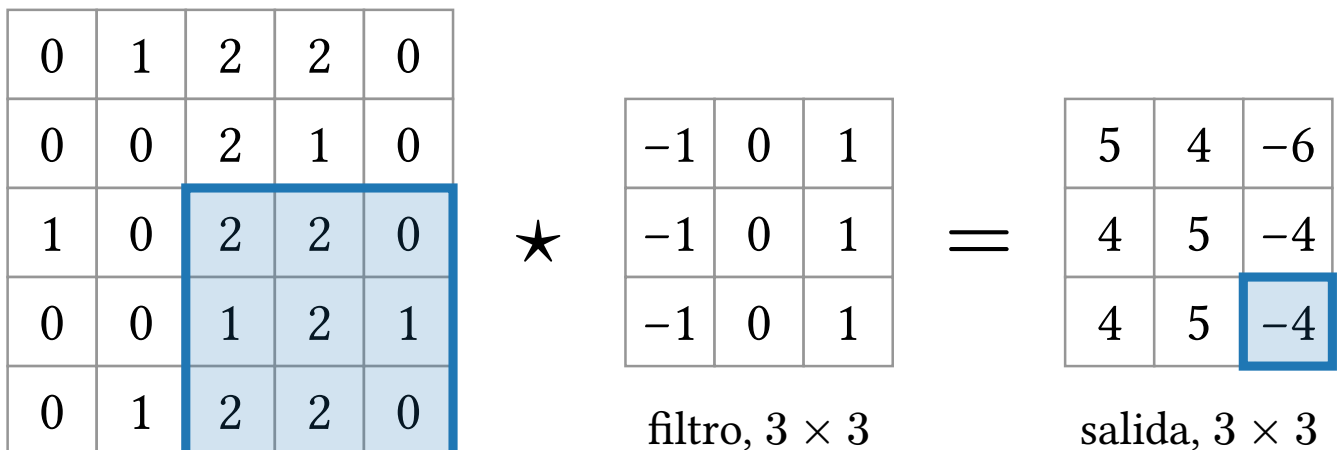

$$1 \cdot (-1) + 2 \cdot 0 + 2 \cdot 1 + 0 \cdot (-1) + 2 \cdot 0 + 1 \cdot 1 + 0 \cdot (-1) + 2 \cdot 0 + 2 \cdot 1 = 4$$

# La convolución




$$2 \cdot (-1) + 2 \cdot 0 + 0 \cdot 1 + 2 \cdot (-1) + 1 \cdot 0 + 0 \cdot 1 + 2 \cdot (-1) + 2 \cdot 0 + 0 \cdot 1 = -6$$

# La convolución




$$2 \cdot (-1) + 2 \cdot 0 + 0 \cdot 1 + 1 \cdot (-1) + 2 \cdot 0 + 1 \cdot 1 + 2 \cdot (-1) + 2 \cdot 0 + 0 \cdot 1 = -4$$

# La convolución en PyTorch

```
X = torch.tensor([[0., 1, 2, 2, 0],
                  [0, 0, 2, 1, 0],
                  [1, 0, 2, 2, 0],
                  [0, 0, 1, 2, 1],
                  [0, 1, 2, 2, 0]])
K = torch.tensor([[-1., 0, 1],
                  [-1, 0, 1],
                  [-1, 0, 1]])

conv = nn.Conv2d(1, 1, kernel_size=3, bias=False)
with torch.no_grad():
    conv.weight[0, 0] = K                 # el filtro, a mano
    out = conv(X.reshape(1, 1, 5, 5))  # (N, C, H, W)

print(out.shape)
print(out[0, 0])
```

Salida

```
torch.Size([1, 1, 3, 3])
tensor([[ 5.,  4., -6.],
        [ 4.,  5., -4.],
        [ 4.,  5., -4.]])
```

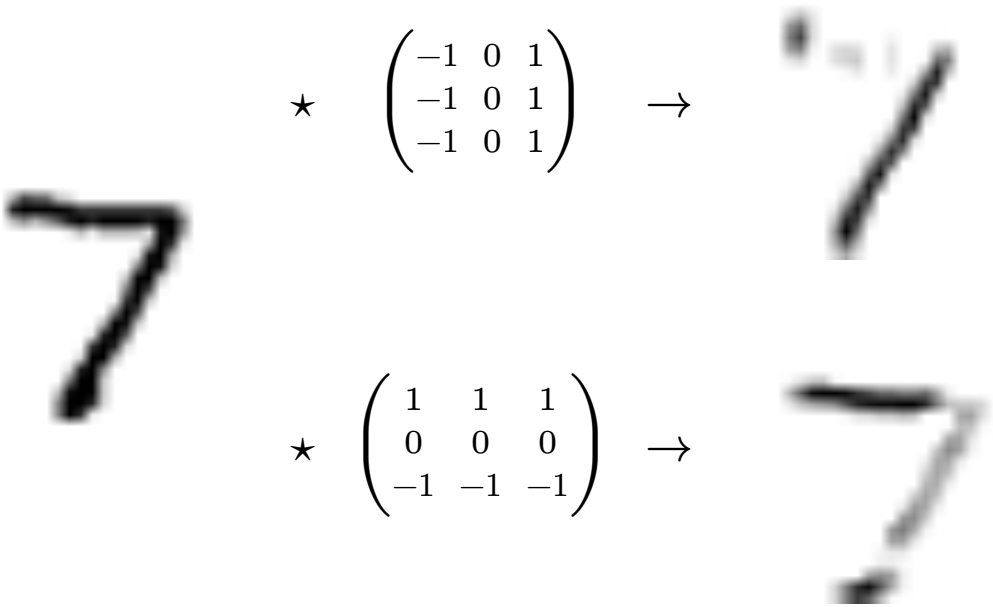
# Ejercicio 3

$$X = \begin{pmatrix} 1 & 2 & 0 & 1 \\ 0 & 1 & 3 & 1 \\ 2 & 0 & 1 & 0 \\ 1 & 1 & 0 & 2 \end{pmatrix}$$

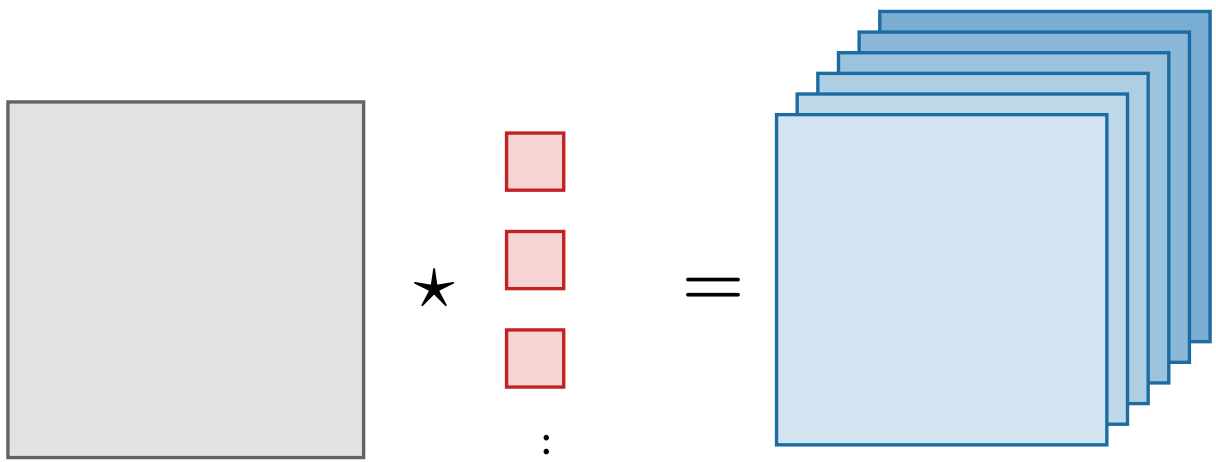
$$K = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

1. ¿De qué tamaño es la salida de $X \star K$? Calcular sus entradas.
2. Aplicar la ReLU a la salida.
3. Si $X$ es un tensor de forma $(1, 1, 4, 4)$, ¿qué forma tiene la salida de `nn.Conv2d(1, 1, kernel_size=2)`?

# Qué detecta un filtro



# Varios filtros: canales



# Varios filtros en PyTorch

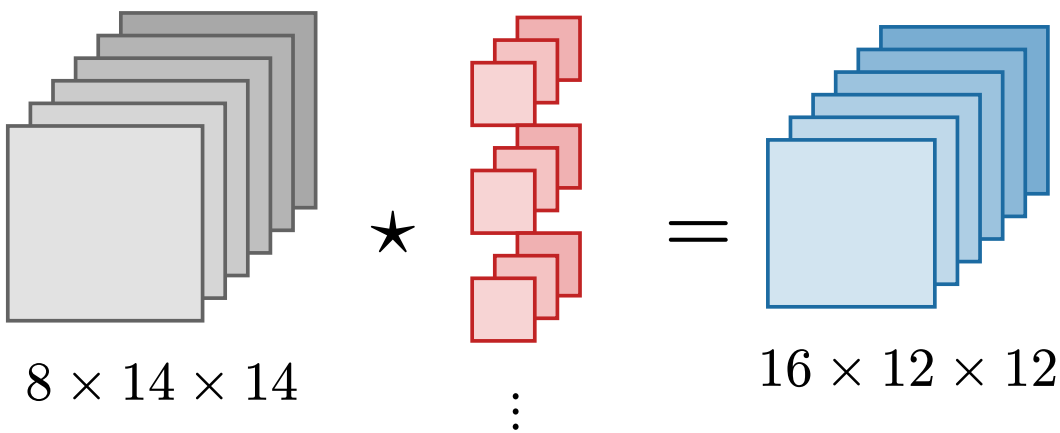
```
x = torch.rand(64, 1, 28, 28)  # 64 imágenes de 1 canal

conv = nn.Conv2d(in_channels=1, out_channels=8, kernel_size=3)
print("pesos:", conv.weight.shape)  # 8 filtros de 1 x 3 x 3
print("sesgos:", conv.bias.shape)  # un sesgo por filtro
print("salida:", conv(x).shape)
print("parámetros:", sum(p.numel() for p in conv.parameters()))
```

Salida

```
pesos: torch.Size([8, 1, 3, 3])
sesgos: torch.Size([8])
salida: torch.Size([64, 8, 26, 26])
parámetros: 80
```

# Entradas con varios canales



# Entradas con varios canales en PyTorch

```
x = torch.rand(64, 8, 14, 14)  # 8 canales de entrada

conv = nn.Conv2d(in_channels=8, out_channels=16, kernel_size=3)
print("pesos:", conv.weight.shape)  # 16 filtros de 8 x 3 x 3
print("salida:", conv(x).shape)
print("parámetros:", sum(p.numel() for p in conv.parameters()))
```

Salida

```
pesos: torch.Size([16, 8, 3, 3])
salida: torch.Size([64, 16, 12, 12])
parámetros: 1168
```

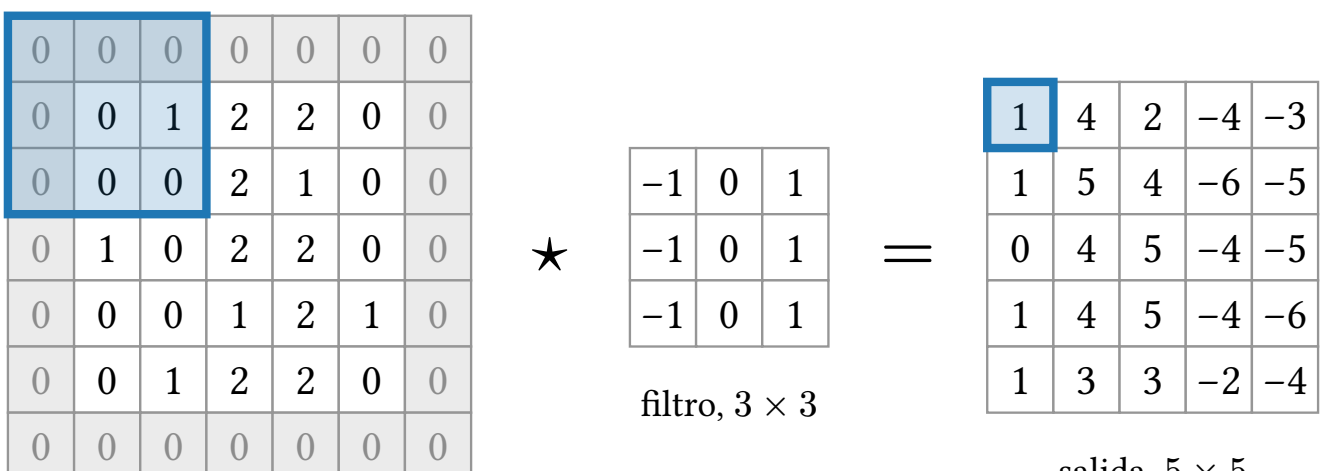
# Número de parámetros

$$(C_{\text{in}} k^2 + 1) C_{\text{out}}$$

`Conv2d(1, 8, 3)`: $(1 \cdot 9 + 1) \cdot 8 = 80$

`Conv2d(8, 16, 3)`: $(8 \cdot 9 + 1) \cdot 16 = 1168$

# Padding




entrada, $7 \times 7$ (con padding)

filtro, $3 \times 3$

salida, $5 \times 5$

# Tamaño de la salida

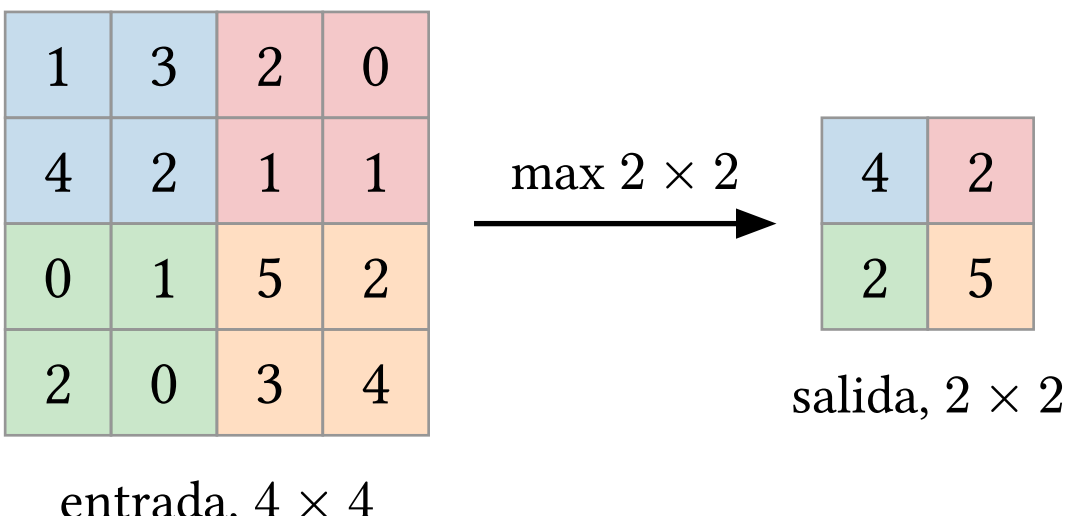
$$m = \left\lfloor \frac{n+2p-k}{s} \right\rfloor + 1$$

```
x = torch.rand(1, 1, 28, 28)
for padding, stride in [(0, 1), (1, 1), (1, 2)]:
    conv = nn.Conv2d(1, 8, kernel_size=3, padding=padding, stride=stride)
    print(f"padding={padding}, stride={stride}:", tuple(conv(x).shape))
```

Salida

```
padding=0, stride=1: (1, 8, 26, 26)
padding=1, stride=1: (1, 8, 28, 28)
padding=1, stride=2: (1, 8, 14, 14)
```

# Max pooling



# Max pooling en PyTorch

```
X = torch.tensor([[1., 3, 2, 0],
                  [4, 2, 1, 1],
                  [0, 1, 5, 2],
                  [2, 0, 3, 4]])
pool = nn.MaxPool2d(kernel_size=2)
print(pool(X.reshape(1, 1, 4, 4))[0, 0])

x = torch.rand(64, 8, 28, 28)
print(x.shape, "->", pool(x).shape)
print("parámetros:", sum(p.numel() for p in pool.parameters()))
```

Salida

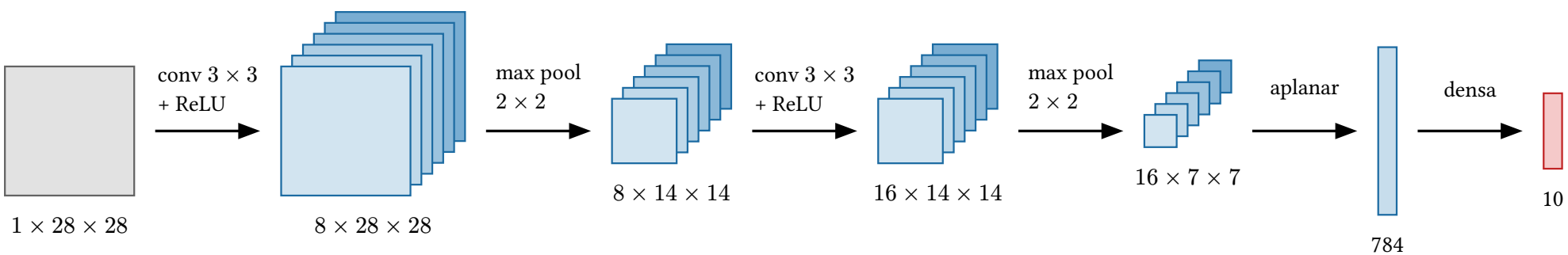
```
tensor([[4., 2.],
        [2., 5.]])
torch.Size([64, 8, 28, 28]) -> torch.Size([64, 8, 14, 14])
parámetros: 0
```

# Ejercicio 4

$$Z = \begin{pmatrix} 2 & 0 & 1 & 5 \\ 3 & 1 & 0 & 2 \\ 0 & 4 & 2 & 2 \\ 1 & 1 & 3 & 0 \end{pmatrix}$$

1. Aplicar max pooling de $2 \times 2$ a $Z$.
2. Un tensor de forma $(64, 16, 14, 14)$ pasa por `nn.MaxPool2d(2)`. ¿Qué forma tiene la salida? ¿Cuántos parámetros tiene la capa?

# Una red convolucional completa



# MyCNN

```
class MyCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 8, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(8, 16, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2)
        self.out = nn.Linear(16 * 7 * 7, 10)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))   # (N, 8, 14, 14)
        x = self.pool(torch.relu(self.conv2(x)))   # (N, 16, 7, 7)
        x = x.flatten(start_dim=1)                 # (N, 784)
        return self.out(x)                       # (N, 10)
```

# Un forward de prueba: las formas

```
model = MyCNN()
x = torch.rand(1, 1, 28, 28)  # una imagen
inventada
print("entrada", tuple(x.shape))
x = torch.relu(model.conv1(x))
print("conv1  ", tuple(x.shape))
x = model.pool(x)
print("pool   ", tuple(x.shape))
x = torch.relu(model.conv2(x))
print("conv2  ", tuple(x.shape))
x = model.pool(x)
print("pool   ", tuple(x.shape))
x = x.flatten(start_dim=1)
print("flatten", tuple(x.shape))
x = model.out(x)
print("densa  ", tuple(x.shape))
```

Salida

```
entrada (1, 1, 28, 28)
conv1   (1, 8, 28, 28)
pool    (1, 8, 14, 14)
conv2   (1, 16, 14, 14)
pool    (1, 16, 7, 7)
flatten (1, 784)
densa   (1, 10)
```

# Un forward de prueba: los parámetros

```python
for nombre, p in model.named_parameters():
    print(f"{nombre:13} {str(tuple(p.shape)):15} {p.numel():5}")
```

Salida

```
conv1.weight  (8, 1, 3, 3)        72
conv1.bias    (8,)                 8
conv2.weight  (16, 8, 3, 3)     1152
conv2.bias    (16,)               16
densa.weight  (10, 784)         7840
densa.bias    (10,)               10
```

# Ejercicio 5

entrada $1 \times 28 \times 28$

`Conv2d(1, 6, kernel_size=5)`

ReLU, `MaxPool2d(2)`

`Conv2d(6, 16, kernel_size=5)`

ReLU, `MaxPool2d(2)`

aplanar

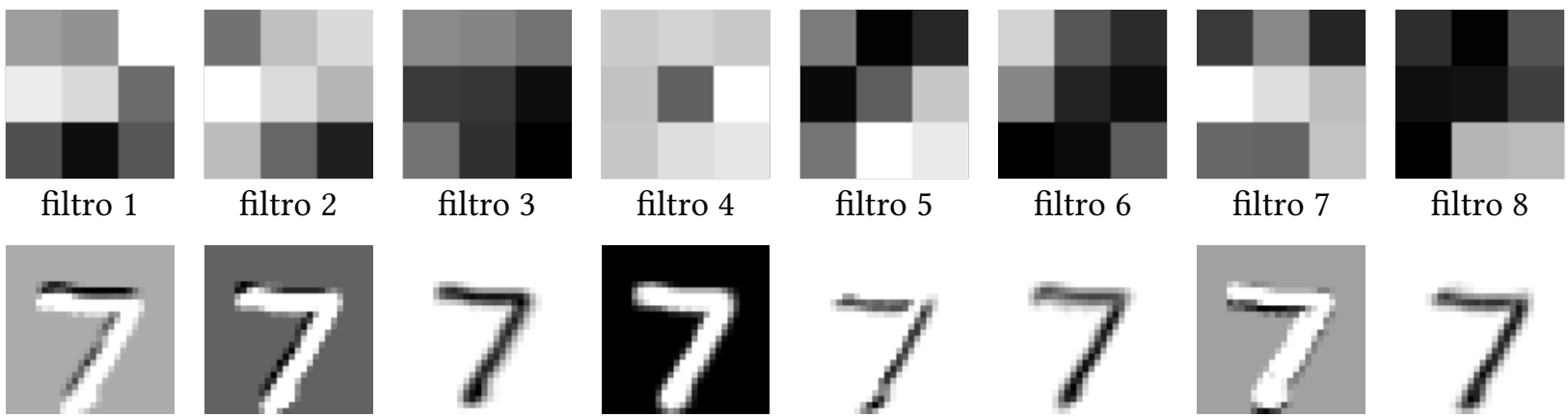
`Linear(?, 10)`

1. Calcular la forma de la salida de cada capa.
2. ¿Cuántas entradas tiene la capa densa?
3. ¿Cuántos parámetros tiene cada capa? ¿Y la red completa?

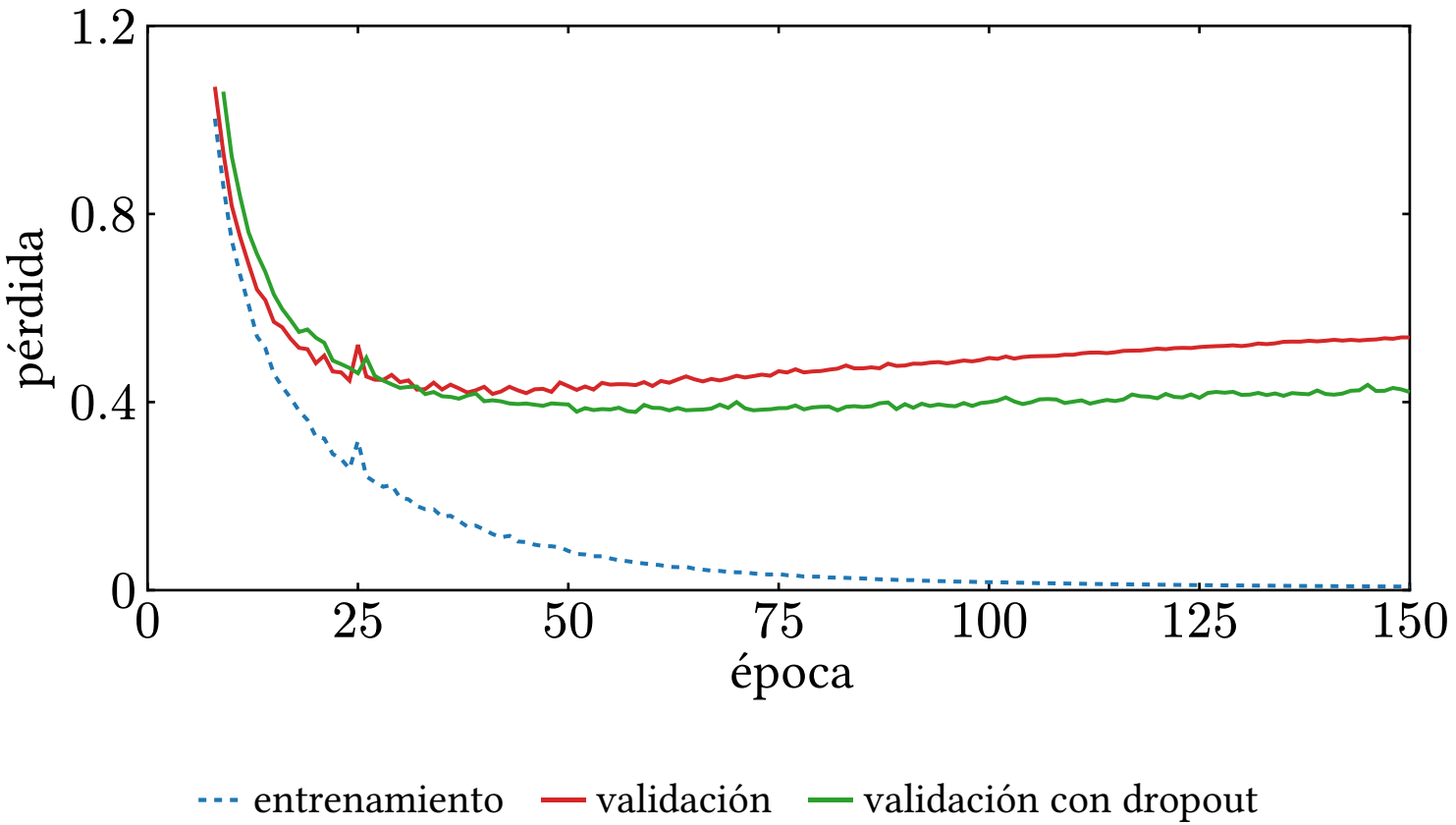
# MLP contra CNN en MNIST

| Red | Parámetros | Validación | Prueba |
|---|---|---|---|
| `MyMLP` | 101 770 | 95.2% | 95.0% |
| `MyCNN` | 9 098 | 97.6% | 97.6% |

# Lo que aprendió la primera capa



# Sobreajuste



# Dropout: `train()` y `eval()`

```
torch.manual_seed(0)
dropout = nn.Dropout(p=0.5)
x = torch.ones(1, 8)

dropout.train()          # entrenamiento: apaga unidades al azar
print(dropout(x))
dropout.eval()           # evaluación: no hace nada
print(dropout(x))
```

Salida

```
tensor([[0., 0., 2., 0., 0., 0., 2., 2.]])
tensor([[1., 1., 1., 1., 1., 1., 1., 1.]])
```

# Weight decay y early stopping

```
# weight decay: regularización L2, como ridge
optimizador = torch.optim.SGD(modelo.parameters(), lr=0.1,
                              weight_decay=1e-4)

# early stopping: guardar los pesos de la mejor época
if perdida_val < mejor_perdida:
    mejor_perdida = perdida_val
    torch.save(modelo.state_dict(), "mejor.pt")
```